Derive topographical variables from point elevations

Gabriel Arellano

2026-11-13

Contents

1 Introduction **2**
1.1 Setup . . . 2

2 From height differences to point elevations **3**

3 Alternative ways of running the code **5**
3.1 From point elevations: all variables . . . 5
3.2 From a DEM: selected variables . . . 8

4 Description of topographical variables **9**
4.1 Elevation . . . 10
4.2 Slope . . . 11
4.3 Aspect, northness and eastness . . . 12
4.4 Topographic position (TPI) . . . 13
4.5 Terrain ruggedness (TRI) . . . 14
4.6 Curvature . . . 15
4.7 Contributing area . . . 16
4.8 Topographic wetness index (TWI) . . . 18
4.9 Downslope distance to drainage . . . 19
4.10 Height above drainage . . . 20

5 Comparison of parameters **22**
5.1 Compare resolutions . . . 22
5.2 Compare smoothing . . . 23
5.3 Compare slope neighbourhoods . . . 24

6 Keep the layers and save GeoTIFF files **25**

References 26

1 Introduction

This tutorial starts with elevations at known coordinates, builds a digital elevation model (DEM), and calculates topographical variables with potential uses in forest ecology. The outputs describe local terrain, exposure, relative position and potential drainage. They can later be assigned to trees, quadrats or other areas.

The general input is a table with `x`, `y` and `z`: two horizontal coordinates and elevation. These values may come from a survey, a data provider or other instruments.

The optional survey section obtains `{x, y, z}` from simulated height differences, using the fgeo2 function `estimate_point_elevations()`. Users who already have elevations can start with the point-input call. Users with a DEM can use the DEM-input call.

`derive_topography()` runs the calculations in one call and returns a named `SpatRaster`. Plotting and permanent file export remain separate. The function uses two packages:

- **`terra`:** the gold standard for raster objects, local terrain calculations, maps and GeoTIFF files. The main object is a `SpatRaster`, with one layer per variable.
- **`whitebox`:** curvature and hydrological calculations. It calls the separate WhiteboxTools program, which needs a one-time installation.

An optional third package, `interp`, provides the linear and spline alternatives described under the point-input call. It is not needed for the default interpolation.

Raster layers and GeoTIFF files are useful in general to store environmental variables. This is useful when topography is later combined with soil, canopy or disturbance data. A separate tutorial will cover combining these sources and extracting values for points or polygons.

1.1 Setup

The example uses within-plot coordinates in metres. Set paths and plotting options here; calculation parameters are declared beside the main call below.

The default paths below work when rendering this tutorial or running from the repository root. Set a full path to fgeo2 when using the code elsewhere.

```
# install.packages(c("terra", "whitebox"))
# install.packages("interp")  # Only for the optional linear or spline interpolation.
# whitebox::install_whitebox()  # Once, to install the WhiteboxTools program.
```

```r
library(terra)
```

```
Warning: package 'terra' was built under R version 4.3.3
```

```
terra 1.8.60
```

```r
path.to.fgeo2 = if(file.exists("../../fgeo2.Rproj")) "../.." else "."
#path.to.fgeo2 = "/Users/salva/Dropbox/PERSONAL/LLC/Oikobit LLC/projects/code-and-statisti

# Temporary by default. Change this to keep the files after the R session.
output.dir = file.path(tempdir(), "fgeo2-topography")
# output.dir = "/full/path/to/save/topography"
dir.create(output.dir, recursive = TRUE, showWarnings = FALSE)
output.dir <- normalizePath(output.dir, winslash = "/")

# Reuse these settings in the maps below.
map.colors = hcl.colors(50, "Viridis")
signed.colors = hcl.colors(51, "Blue-Red 3", rev = TRUE)
aspect.colors = hcl.colors(36, "Dynamic")
map.margins = c(3, 3, 2, 4)
knitr::opts_chunk$set(fig.width = 6, fig.height = 5)
source(file.path(path.to.fgeo2, "06_topography_habitat/R/new/topography-variables.R"))
```

2 From height differences to point elevations

Skip this section if a provider has already supplied an `{x, y, z}` table. The rest of the tutorial does not depend on how those elevations were obtained.

For this example, positive `dz` means that the destination (point j) is higher than the origin (point i). The reference point sets the elevation datum. See the solver tutorial for the survey and its checks. The solver also needs `Matrix` and `igraph` R packages.

```r
records <- read.csv(file.path(path.to.fgeo2, "data_examples/topography-survey.csv"))
reference.xy = c(0, 0)  # {x,y} coordinates of a reference point
reference.z = 371 # height or z coordinate of a reference point

survey <- data.frame(
    x_i = records$from_x_m,
    y_i = records$from_y_m,
```

```
    x_j = records$to_x_m,
    y_j = records$to_y_m,
    dz = records$height_difference_m
)

source(file.path(path.to.fgeo2, "06_topography_habitat/R/new/topography-solver.R"))
elevations <- estimate_point_elevations(survey, reference.xy, reference.z)
head(elevations)
```

```
  x   y        z
1 0   0 371.0000
2 0  20 371.2909
3 0  40 371.5931
4 0  60 371.8797
5 0  80 372.1822
6 0 100 372.4800
```

```
# General entry point when elevations come from elsewhere:
# elevations <- read.csv("/full/path/to/point-elevations.csv")
# names(elevations) must include x, y and z, in compatible metre units.
```

```
elevation.points <- vect(elevations, geom = c("x", "y"), crs = "local")
plot(elevation.points, "z", pch = 15, cex = 2, col = map.colors,
     main = "Point elevations (m)", axes = TRUE, mar = map.margins)
points(elevations[,c("x", "y")], col = "white", pch = "·")
```

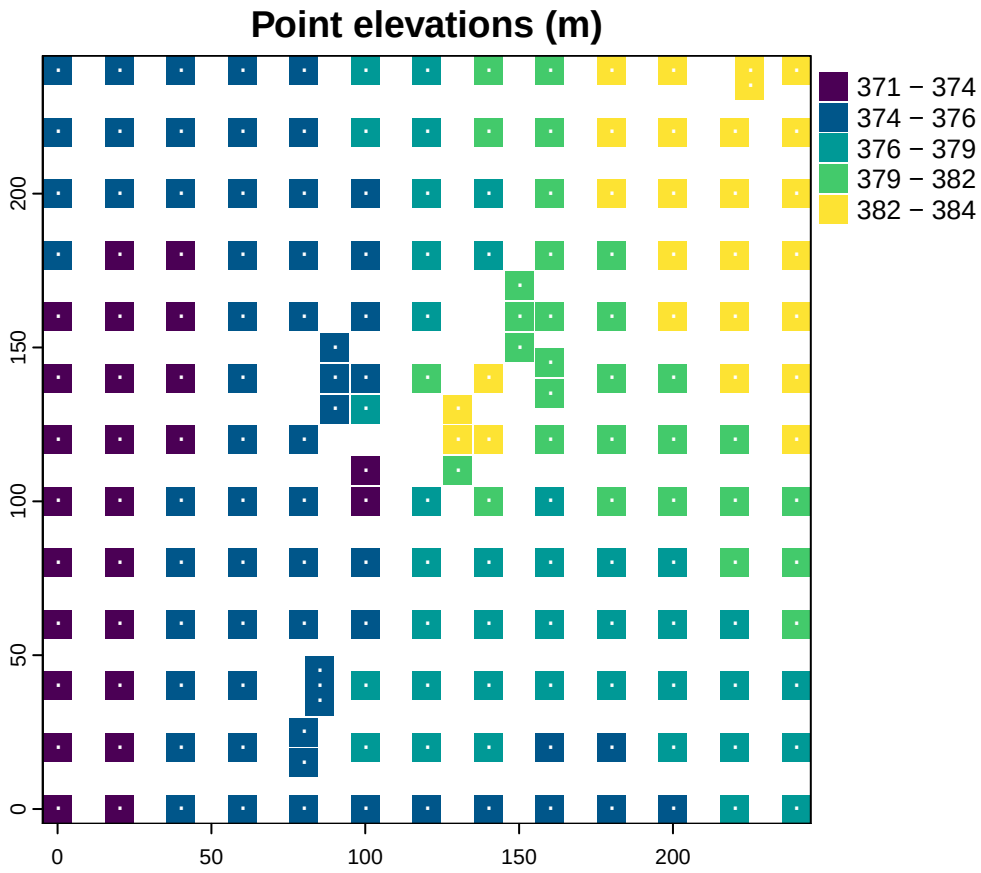


Figure 1: Point elevations used to build the DEM. No regular input grid is required.

3 Alternative ways of running the code

3.1 From point elevations: all variables

The shortest call is `topography <- derive_topography(xyz = elevations)`. It builds a DEM and returns all variables by default. The example below declares the parameters explicitly so they are easy to change. This `topography` object is used throughout the rest of the tutorial.

The function accepts an elevation table or a matrix with `x`, `y` and `z`. Incomplete rows are omitted with a warning. Repeated identical points are accepted; conflicting elevations at the same coordinates must be resolved before interpolation.

The main choices are:

- **Resolution:** `cell.size` sets the output spacing. A 5 m raster does not create 5 m measured detail from a survey taken mostly at 20 m intervals.
- **Interpolation:** the default inverse-distance weighting (IDW) gives nearby points more weight. Its search radius, distance power and smoothing parameter are declared below. `linear` connects the elevations with planar triangles; `akima` gives a smoother surface, which may overshoot the observed range. The last two options need interp. `bilinear` is available when the input points form a complete regular grid.
- **Smoothing:** `smoothing.cells` sets a moving mean applied before the derivatives. At 5 m resolution, a width of 3 covers 15 m. Use 1 to keep the interpolated DEM unchanged. The function defaults to 1, with no smoothing. This example explicitly uses 3. Compare choices before interpreting small ridges or depressions.

Raster coordinates refer to cell centres. A 5 m raster over 0–240 m has centres at 2.5, 7.5, …, 237.5 m. Interpolating at those centres keeps the raster boundary at the plot boundary.

Raster resolution, smoothing and neighbourhood choices are compared near the end of the tutorial.

```
plot.extent = c(0, 240, 0, 240)  # xmin, xmax, ymin, ymax, in metres.
cell.size = 5
interpolation = "idw"          # "idw", "linear", "akima" or "bilinear".
search.radius = 60             # IDW search radius, in metres.
idw.power = 2                  # Distance weighting: larger values favour nearer points.
idw.smooth = 0                 # IDW distance-smoothing parameter, in metres.
min.points = 3                # Minimum points within the IDW search radius.
max.points = 12               # Maximum points used for IDW.
slope.neighbors = 8           # 4 or 8 neighbours for slope and aspect.
smoothing.cells = 3            # Odd window width; 1 means no smoothing.
position.radius = 20           # TPI neighbourhood radius, in metres.
flow.preparation = "breach"   # "breach", "fill" or "none" for an already prepared DEM.
breach.distance.cells = 20     # Maximum search distance for drainage preparation.
channel.area.m2 = 1000         # Illustrative channel threshold; check in the field.

# Build the DEM and calculate all variables.
topography <- derive_topography(
    xyz = elevations, resolution = cell.size, extent = plot.extent, crs = "local",
    interpolation = interpolation, search.radius = search.radius,
    idw.power = idw.power, idw.smooth = idw.smooth,
    min.points = min.points, max.points = max.points,
    smoothing.cells = smoothing.cells, neighbors = slope.neighbors,
    position.radius = position.radius,
```

```
    flow.preparation = flow.preparation,
    breach.distance.cells = breach.distance.cells, channel.area.m2 = channel.area.m2
)

dem <- topography[["elevation_m"]]
names(topography)
```

```
 [1] "elevation_m"             "slope_deg"
 [3] "aspect_deg"              "northness"
 [5] "eastness"                "tpi_m"
 [7] "tri_m"                   "plan_curvature_per_m"
 [9] "profile_curvature_per_m" "contributing_area_m2"
[11] "twi"                     "drainage_distance_m"
[13] "height_above_drainage_m"
```

The function leaves cells outside the observed points' convex hull as `NA`. Smoothing preserves those missing cells and uses available neighbours at the boundary. Check the DEM for gaps and interpolation artefacts before interpreting the layers. If linear interpolation leaves interior gaps, use IDW or `bilinear` for a complete regular input grid. For larger sites, supply the terrain surrounding the plot and crop the outputs afterwards.

```
plot(dem, col = map.colors, main = "Elevation (m)",
     axes = TRUE, mar = map.margins)
points(elevations[, c("x", "y")], pch = 16, cex = 0.2)
```

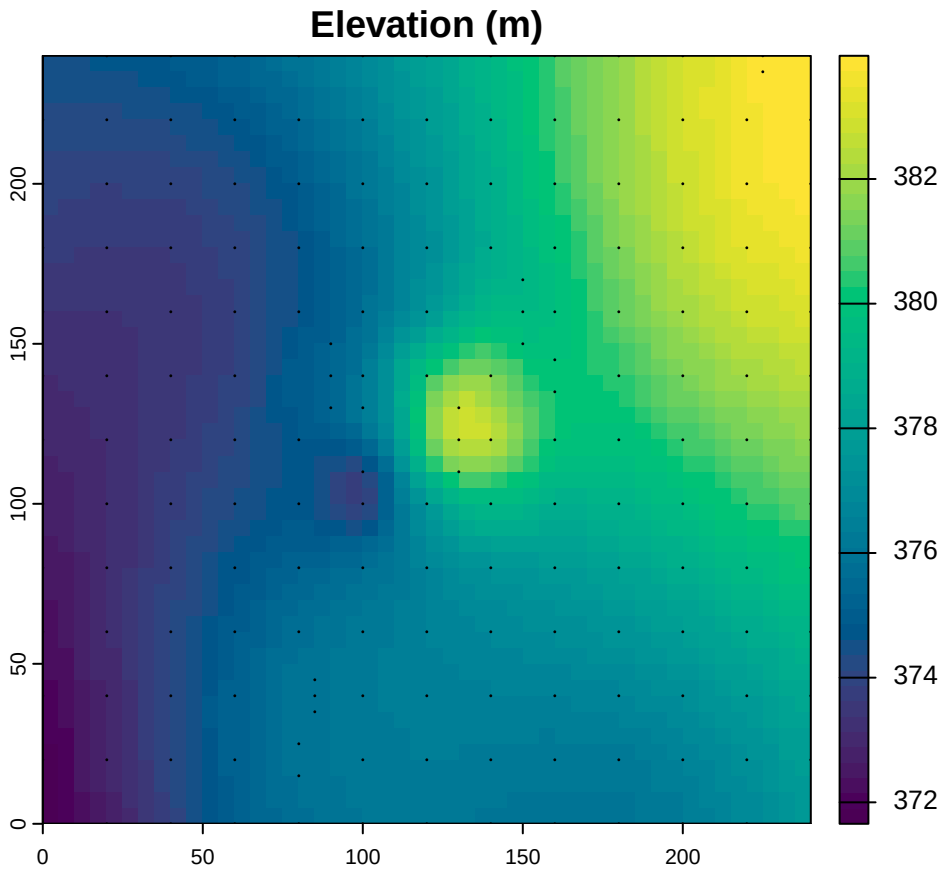


Figure 2: Interpolated DEM after the selected smoothing. Points show the input locations.

3.2 From a DEM: selected variables

Alternatively, supply a DEM and request only the variables needed. A supplied DEM bypasses interpolation and retains its grid and CRS. Smoothing defaults to 1, so its elevations are also unchanged unless smoothing is requested. The workflow requires square raster cells and compatible horizontal and vertical metre units. Do not label longitude and latitude coordinates as metres.

```
# Reuse the example DEM. To use your own file, replace this line:
dem <- topography[["elevation_m"]]
# dem <- rast("/full/path/to/elevation.tif")
```

```r
local.topography <- derive_topography(
    dem = dem, variables = c("slope", "tpi", "tri")
)
names(local.topography)
```

```
[1] "elevation_m" "slope_deg"   "tpi_m"       "tri_m"
```

Elevation is always included. This call uses the default calculation parameters; add arguments as in the previous call to change them. Requesting only these local variables does not need WhiteboxTools. Available variable names and parameters are documented at the top of `topography-variables.R`.

4 Description of topographical variables

The variables below are complementary descriptions of the same terrain. They are often correlated; calculating them all does not mean that all should enter the same statistical model. Each plot uses a layer from the main `topography` object.

- **Elevation:** height relative to the chosen datum.
- **Slope:** terrain inclination.
- **Aspect, northness and eastness:** orientation of the slope, with two continuous components for analysis.
- **Topographic position (TPI):** height relative to a stated neighbourhood; a local description of ridges and depressions.
- **Terrain ruggedness (TRI):** local elevation differences.
- **Plan and profile curvature:** changes in surface direction and slope.
- **Contributing area:** terrain supplying runoff to each location.
- **Topographic wetness index (TWI):** potential for water accumulation from contributing area and slope.
- **Downslope distance to drainage:** distance along the drainage route.
- **Height above drainage:** vertical separation from the drainage reached downhill.

Recent forest studies use aspect components and ruggedness (McNichol et al. 2024; Freund et al. 2021), and wetness indices (Zhang et al. 2025). Curvature and drainage position also have applications to forest structure and floristic composition (Detto et al. 2013; Schietti et al. 2014).

Spatial scale is part of each variable. DEM resolution, smoothing and neighbourhood size can change the results (Detto et al. 2013; Muscarella et al. 2020). Blank boundary cells are retained where the full local neighbourhood is unavailable.

The old CTFS/ctfs/fgeo quadrat summary returned mean elevation, slope and convexity for regular square quadrats (20 m by default, configurable). This function accepts irregular input points and calculates variables on a raster with adjustable cell size. See the evaluation for details.

4.1 Elevation

Elevation is already in the DEM. It can describe broad gradients within a plot, although it does not directly measure soil moisture or soil fertility. An arbitrary elevation datum is sufficient for the relative terrain calculations below. The old CTFS/fgeo code used the mean of four quadrat corners; here elevation is interpolated at a raster cell centre, with optional smoothing.

```
elevation <- topography[["elevation_m"]]
plot(elevation, col = map.colors, main = "Elevation (m)",
     axes = TRUE, mar = map.margins)
```

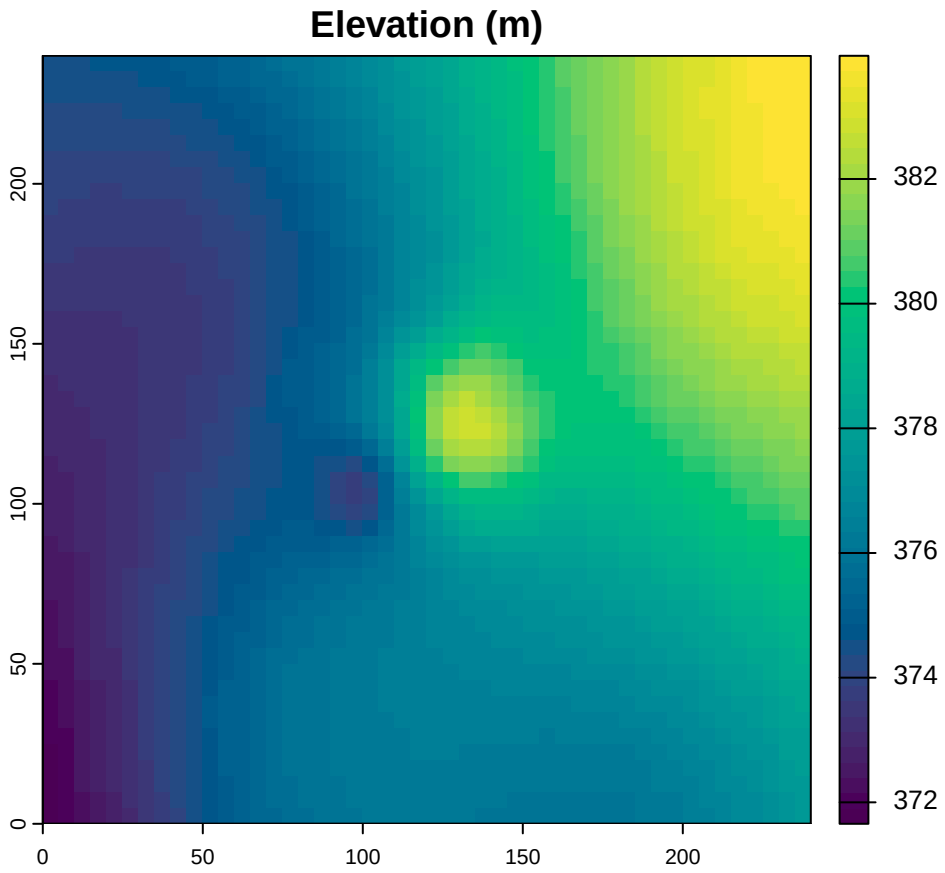


Figure 3: **?(caption)**

4.2 Slope

Slope is the inclination from horizontal, in degrees. It describes steepness and is often used when analysing forest structure or disturbance. The example uses the eight neighbouring cells; `neighbors = 4` is an alternative. The old CTFS/fgeo code instead averaged the inclinations of four planes, each through three quadrat corners. Both raster methods differ from that calculation.

```
slope <- topography[["slope_deg"]]
plot(slope, col = map.colors, main = "Slope (degrees)",
     axes = TRUE, mar = map.margins)
```

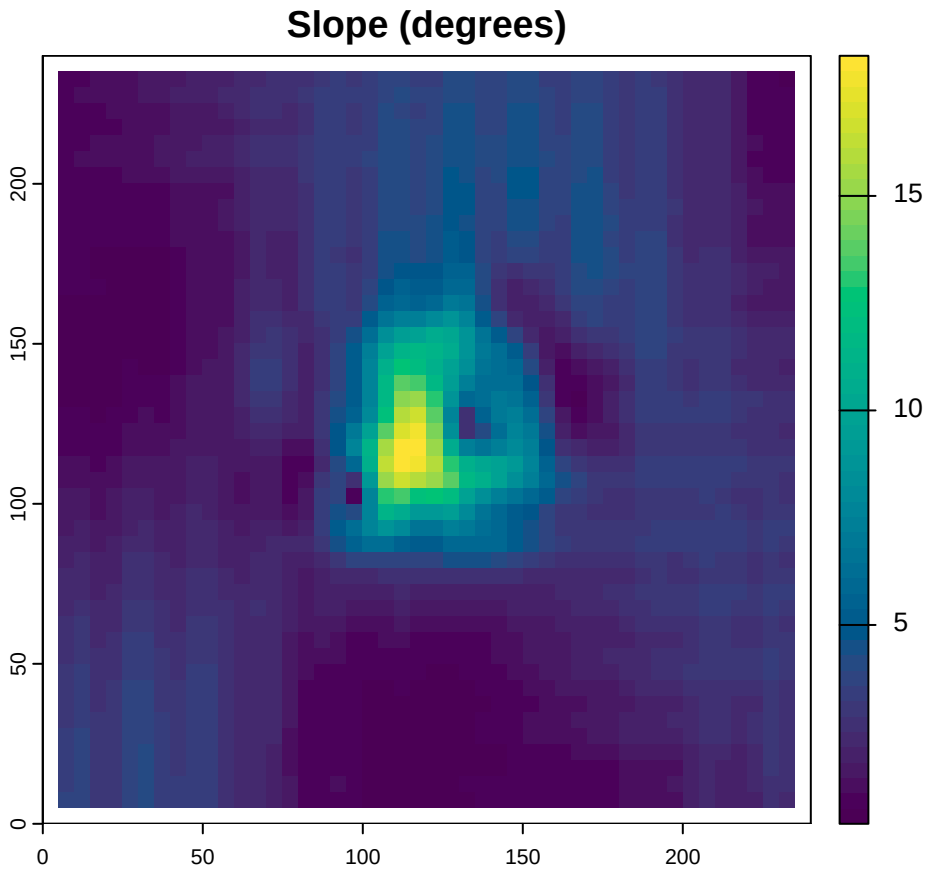


Figure 4: **?(caption)**

4.3 Aspect, northness and eastness

Aspect is the downhill direction, clockwise from north. It can represent differences in exposure, as in the Niobrara ForestGEO study of microclimate (McNichol et al. 2024). On our local grid, this interpretation assumes that positive `y` points north.

Northness and eastness avoid the discontinuity between 359 and 0 degrees: they are the cosine and sine of aspect. Flat cells have no meaningful aspect and are left blank. The old CTFS `calc.gradient()` calculated directions for eight triangular facets rather than a single raster aspect and its northness/eastness components.

```r
aspect <- topography[["aspect_deg"]]
northness <- topography[["northness"]]
eastness <- topography[["eastness"]]

settings <- par(mfrow = c(1, 3))
plot(aspect, col = aspect.colors, range = c(0, 360), main = "Aspect (degrees)", mar = map.
plot(northness, col = signed.colors, range = c(-1, 1), main = "Northness", mar = map.margi
plot(eastness, col = signed.colors, range = c(-1, 1), main = "Eastness", mar = map.margins
par(settings)
```

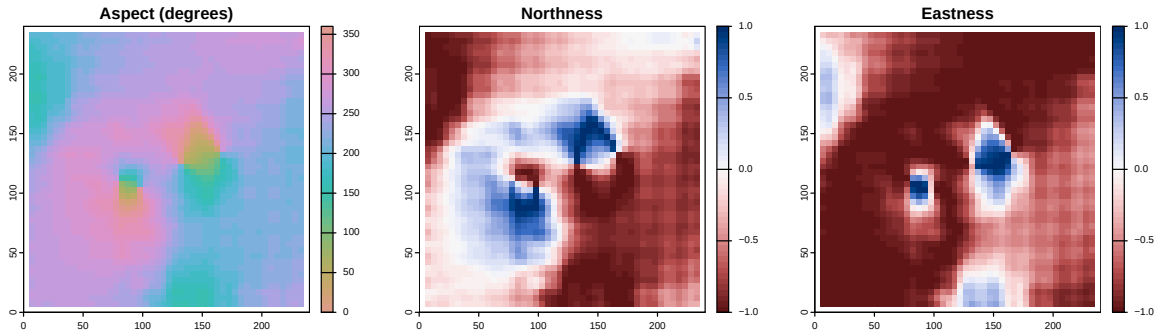


Figure 5: **?(caption)**

4.4 Topographic position (TPI)

TPI is focal elevation minus the mean elevation of its neighbours. Positive values indicate locally higher ground; negative values indicate depressions. The neighbourhood below is a square extending `position.radius` metres from the focal cell centre, rounded up to a whole number of cells.

The old CTFS/fgeo convexity was a quadrat's mean elevation minus the mean of its eight surrounding quadrats. At the perimeter, the default calculation used measured centre elevation minus corner mean. Here, TPI uses raster cells, an explicit radius and no estimate for incomplete neighbourhoods.

```r
tpi <- topography[["tpi_m"]]
tpi.limit <- global(abs(tpi), "max", na.rm = TRUE)[1, 1]
plot(tpi, col = signed.colors, range = c(-tpi.limit, tpi.limit),
```

```
     main = "Topographic position (m)", axes = TRUE, mar = map.margins)
```

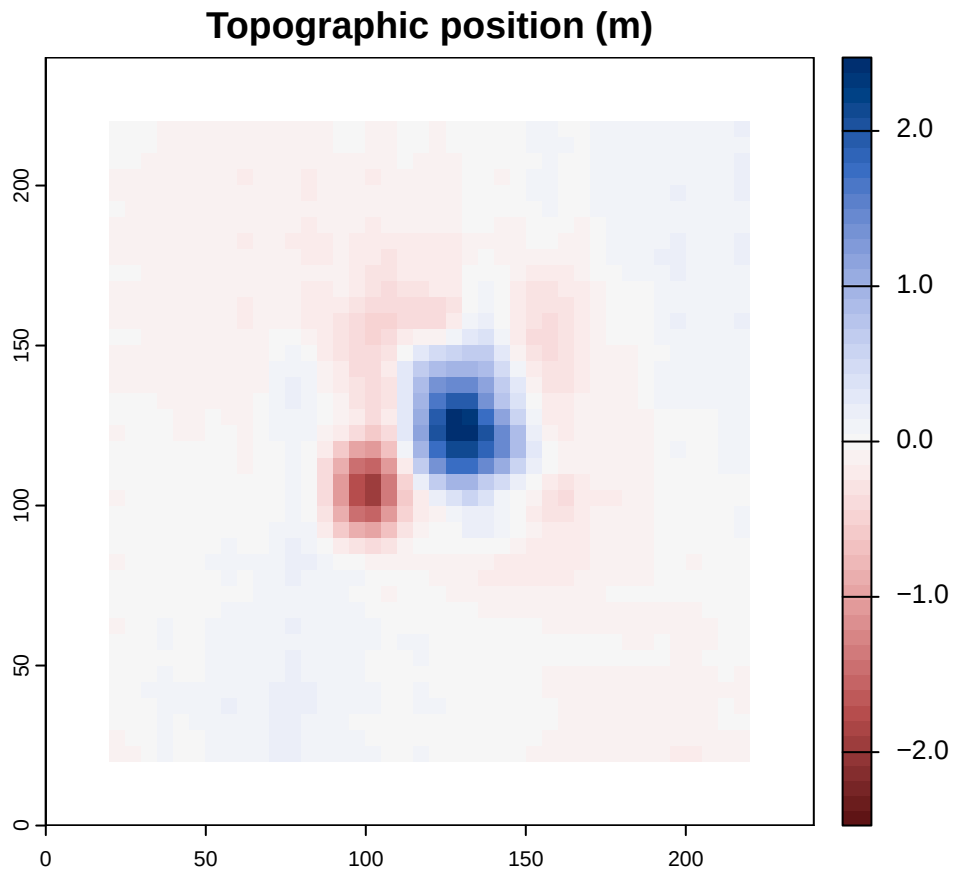


Figure 6: **?(caption)**

4.5 Terrain ruggedness (TRI)

This TRI is the mean absolute elevation difference between a cell and its eight neighbours, in metres. Freund et al. (2021) used this definition in a study of forest recovery after landslides. It measures local relief and can be strongly related to slope.

```
tri <- topography[["tri_m"]]
plot(tri, col = map.colors, main = "Terrain ruggedness (m)",
     axes = TRUE, mar = map.margins)
```

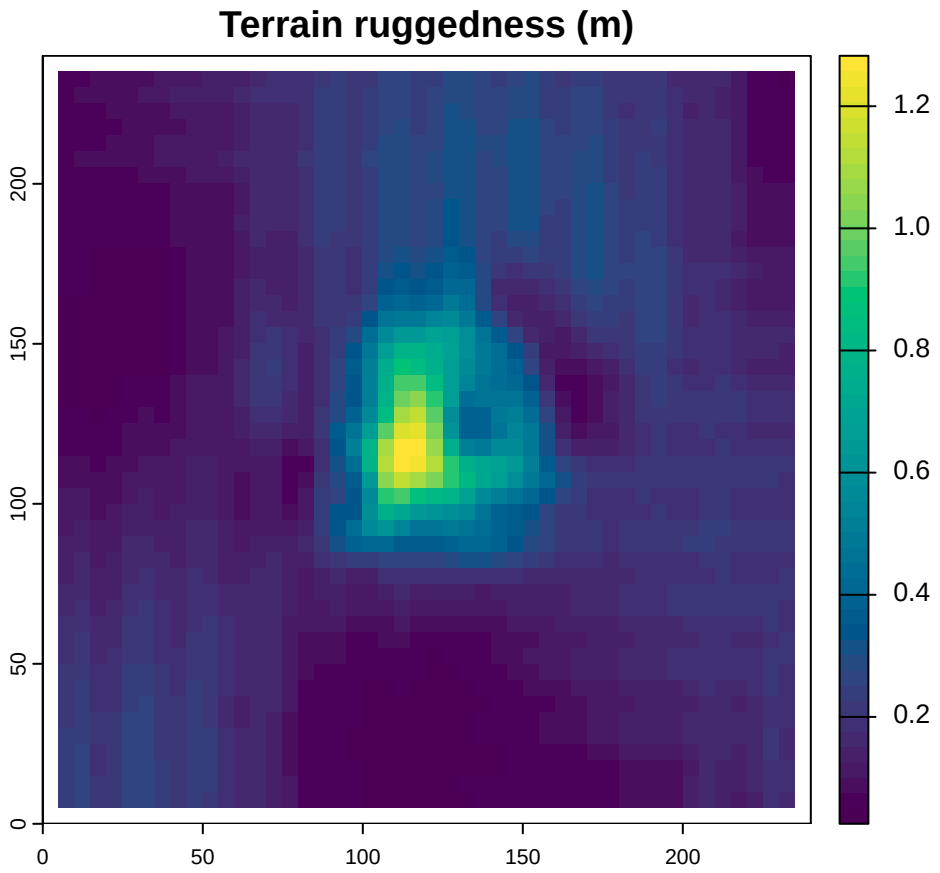


Figure 7: **?(caption)**

4.6 Curvature

Plan curvature describes convergence or divergence across the slope. Profile curvature describes how steepness changes downhill. These are second derivatives of elevation and are particularly sensitive to interpolation and smoothing. Detto et al. (2013) used several curvature measures to describe forest canopy structure in Panama.

The WhiteboxTools 2.4.0 implementation used here reports curvature in inverse metres. Positive plan curvature indicates divergence; positive profile curvature indicates acceleration of downhill flow. The two signs describe different things. These differ from CTFS/fgeo convexity, which compared a quadrat with its neighbours. The calculation uses a 5 by 5 cell fit; the function masks the incomplete boundary.

```r
plan.curvature <- topography[["plan_curvature_per_m"]]
profile.curvature <- topography[["profile_curvature_per_m"]]
plan.limit <- global(abs(plan.curvature), "max", na.rm = TRUE)[1, 1]
profile.limit <- global(abs(profile.curvature), "max", na.rm = TRUE)[1, 1]

settings <- par(mfrow = c(1, 2))
plot(plan.curvature, col = signed.colors, range = c(-plan.limit, plan.limit),
     main = "Plan curvature (1/m)", mar = map.margins)
plot(profile.curvature, col = signed.colors, range = c(-profile.limit, profile.limit),
     main = "Profile curvature (1/m)", mar = map.margins)
par(settings)
```

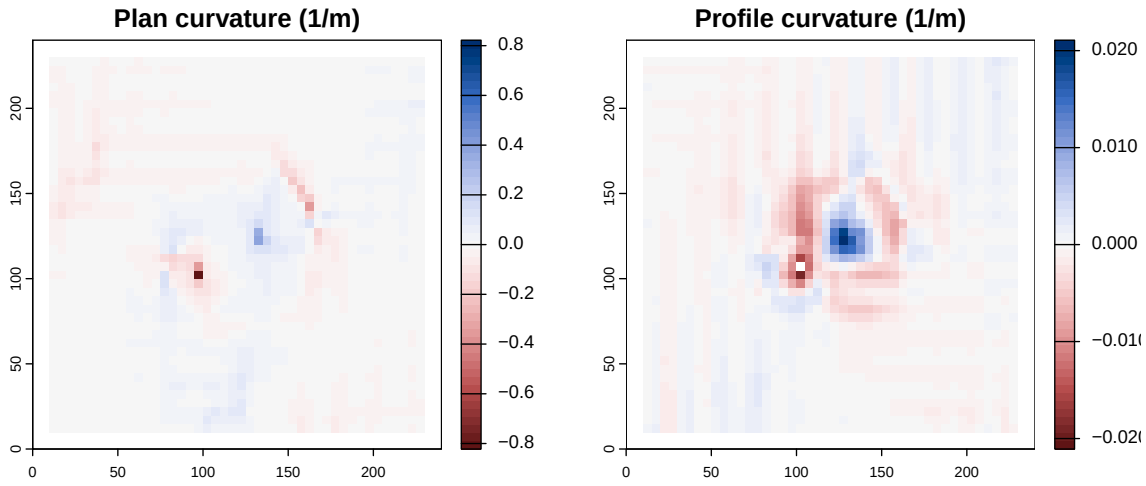


Figure 8: **?(caption)**

4.7 Contributing area

Contributing area describes how much upslope terrain supplies flow to a cell. It is an input to TWI. The function uses D-infinity accumulation, which can divide flow between neighbouring directions. The specific contributing area (SCA) is area per unit contour width, in metres; multiplying by cell width gives the corresponding area in square metres. The old CTFS/fgeo quadrat summary did not include contributing area or the following flow-based variables.

Flow routing needs a draining surface. By default, a separate copy of the DEM is prepared by breaching depressions, with filling where breaching fails. `flow.preparation = "fill"`

instead raises depressions; use `"none"` for a DEM already prepared for routing. Inspect these choices where depressions are real features. The earlier local terrain layers still use the original DEM.

The example covers only the plot. It omits water arriving from outside. The following drainage maps demonstrate the workflow, but are not complete catchment estimates. For ecological analysis, use surrounding terrain and delineated drainage appropriate to the site; Muscarella et al. (2020) explicitly tested DEM extent and resolution.

```
contributing.area <- topography[["contributing_area_m2"]]
plot(log10(contributing.area), col = map.colors,
     main = "Contributing area: log10(m2)", axes = TRUE, mar = map.margins)
```

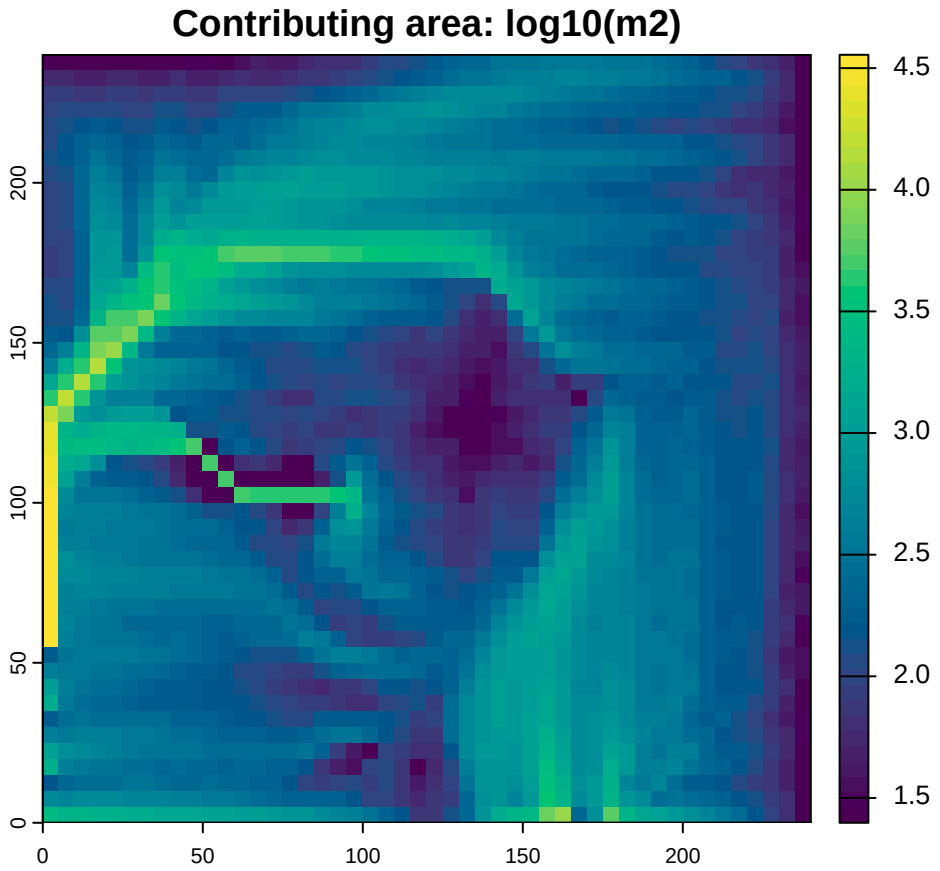


Figure 9: **?(caption)**

4.8 Topographic wetness index (TWI)

TWI combines specific contributing area and local slope: `log(SCA / tan(slope))`. Higher values indicate greater topographic potential for water accumulation. It is a proxy, not measured soil moisture. Zhang et al. (2025) used TWI to describe tropical forest canopy dynamics.

The function supplies Whitebox with **unlogged SCA and slope in degrees**. Whitebox converts the angle internally. Zero-slope cells become `NA`. Here, slope comes from the same DEM used for the local terrain variables; only the flow-routing surface was conditioned.

```
twi <- topography[["twi"]]
plot(twi, col = map.colors, main = "Topographic wetness index",
     axes = TRUE, mar = map.margins)
```

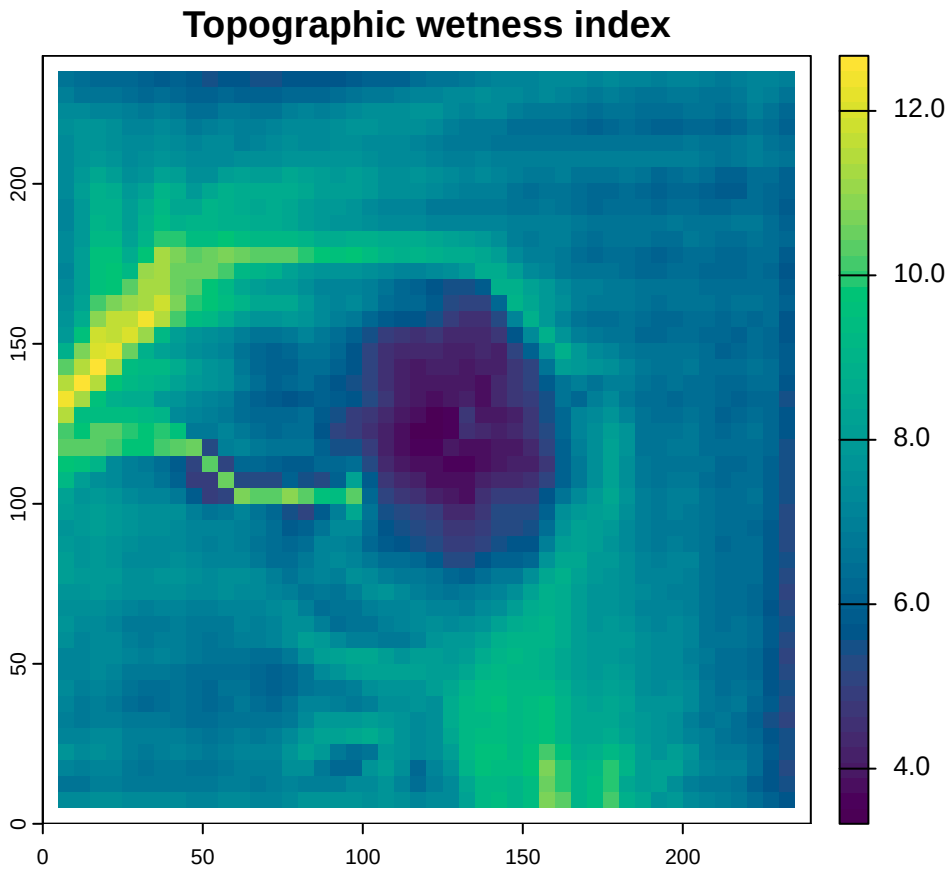


Figure 10: **?(caption)**

4.9 Downslope distance to drainage

This is distance along the downhill flow path to a channel, not straight-line distance to the closest channel. Detto et al. (2013) related flow distance to canopy height.

A channel network must first be defined. The example uses a D8 contributing-area threshold, in square metres. Change `channel.area.m2` to reflect site hydrology and check it against mapped channels. This synthetic network does not identify observed or perennial streams.

```
drainage.distance <- topography[["drainage_distance_m"]]
plot(drainage.distance, col = map.colors, main = "Downslope distance to drainage (m)",
     axes = TRUE, mar = map.margins)
```

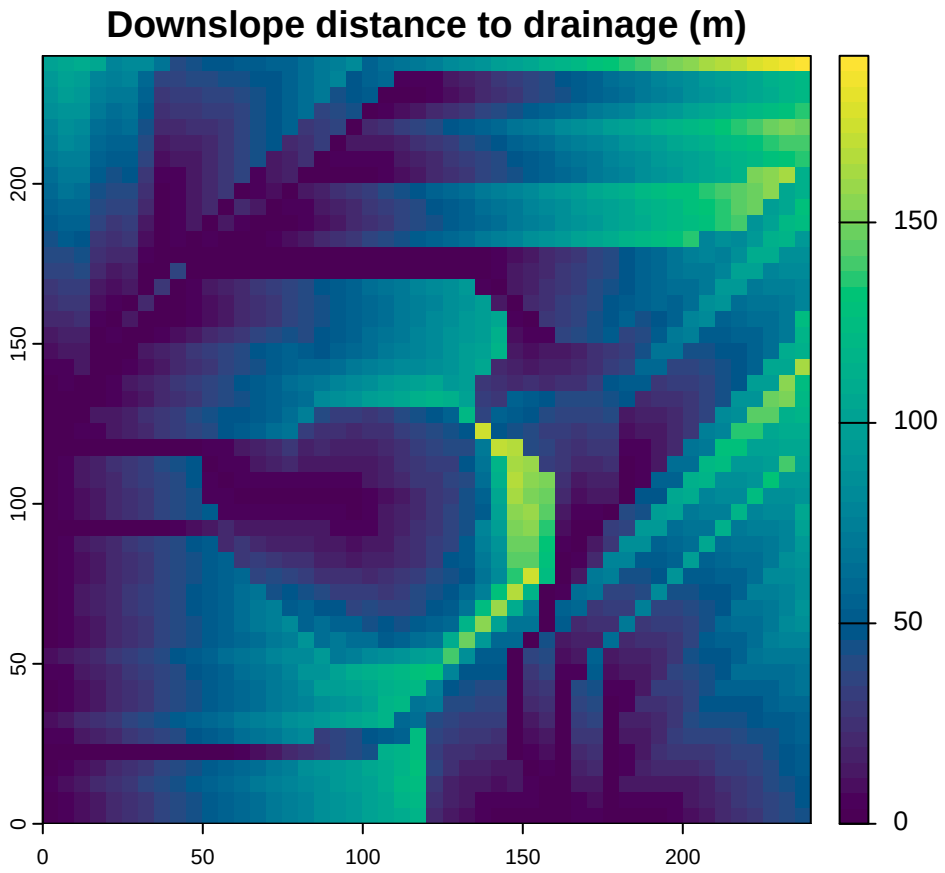


Figure 11: **?(caption)**

The D8 network routes flow to one neighbour; it is used consistently for the drainage distance and height below. TWI uses the distributed D-infinity accumulation. These are separate, visible choices. Cells whose route leaves the DEM without reaching a defined channel can remain blank.

4.10 Height above drainage

This is elevation relative to the channel reached along the downhill flow path. It is often called HAND: height above the nearest drainage. Here, "nearest" refers to the drainage route, not the shortest map distance. Schietti et al. (2014) used HAND to describe floristic composition in an Amazonian forest.

HAND can distinguish terrain near valley bottoms from higher slopes. It is not a direct measurement of groundwater depth. The channel threshold and the extent of the DEM remain important.

```
hand <- topography[["height_above_drainage_m"]]
plot(hand, col = map.colors, main = "Height above drainage (m)",
     axes = TRUE, mar = map.margins)
```

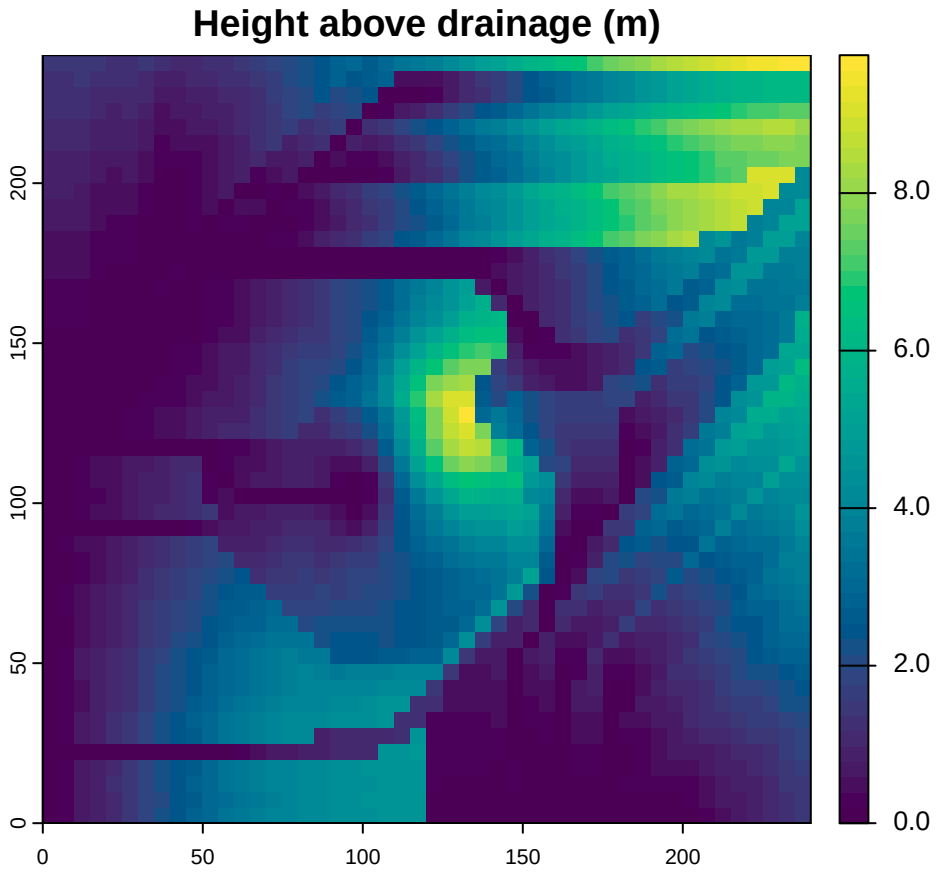


Figure 12: **?(caption)**

5 Comparison of parameters

5.1 Compare resolutions

These calls use the same point data and interpolation settings, without smoothing. Only raster resolution changes. Both maps use the same colour range; compare patterns rather than expecting cell-by-cell agreement.

```
fine <- derive_topography(
    xyz = elevations, variables = "slope", resolution = 5, extent = plot.extent,
    interpolation = interpolation, search.radius = search.radius,
    idw.power = idw.power, idw.smooth = idw.smooth,
    min.points = min.points, max.points = max.points, neighbors = slope.neighbors
)

coarse <- derive_topography(
    xyz = elevations, variables = "slope", resolution = 10, extent = plot.extent,
    interpolation = interpolation, search.radius = search.radius,
    idw.power = idw.power, idw.smooth = idw.smooth,
    min.points = min.points, max.points = max.points, neighbors = slope.neighbors
)

slope.limit <- max(global(fine[["slope_deg"]], "max", na.rm = TRUE)[1, 1],
                   global(coarse[["slope_deg"]], "max", na.rm = TRUE)[1, 1])

settings <- par(mfrow = c(1, 2))
plot(fine[["slope_deg"]], col = map.colors, range = c(0, slope.limit),
     main = "5 m cells", mar = map.margins)
plot(coarse[["slope_deg"]], col = map.colors, range = c(0, slope.limit),
     main = "10 m cells", mar = map.margins)
par(settings)
```

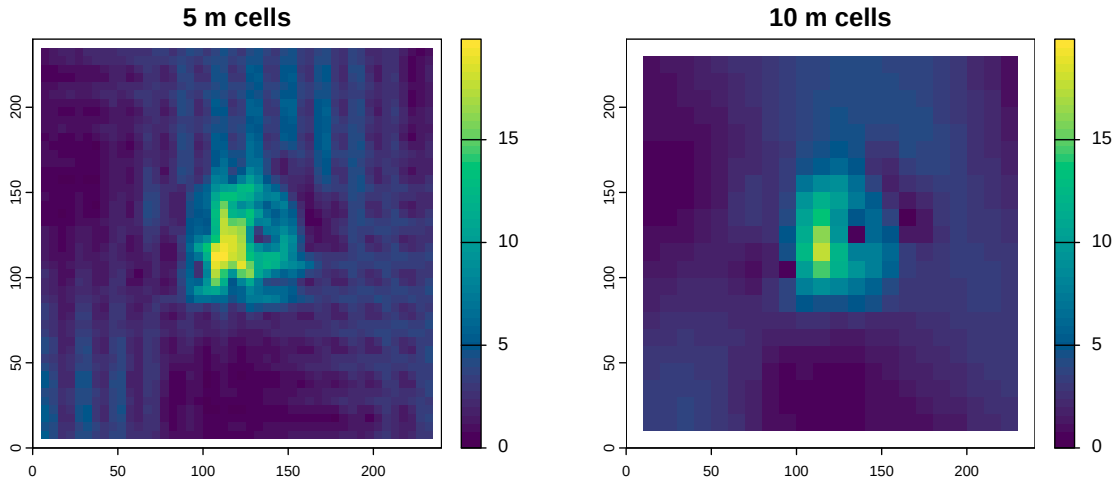


Figure 13: Slope calculated at 5 m and 10 m resolution, with no smoothing.

5.2 Compare smoothing

Reuse the unsmoothed 5 m DEM, so interpolation does not change. The second calculation applies a 5 by 5 cell mean, covering 25 m. Smoothing can reduce local noise, but it can also remove real terrain features.

```
smoothed <- derive_topography(
    dem = fine[["elevation_m"]], variables = "slope",
    smoothing.cells = 5, neighbors = slope.neighbors
)

slope.limit <- max(global(fine[["slope_deg"]], "max", na.rm = TRUE)[1, 1],
                   global(smoothed[["slope_deg"]], "max", na.rm = TRUE)[1, 1])

settings <- par(mfrow = c(1, 2))
plot(fine[["slope_deg"]], col = map.colors, range = c(0, slope.limit),
     main = "No smoothing", mar = map.margins)
plot(smoothed[["slope_deg"]], col = map.colors, range = c(0, slope.limit),
     main = "25 m smoothing", mar = map.margins)
par(settings)
```

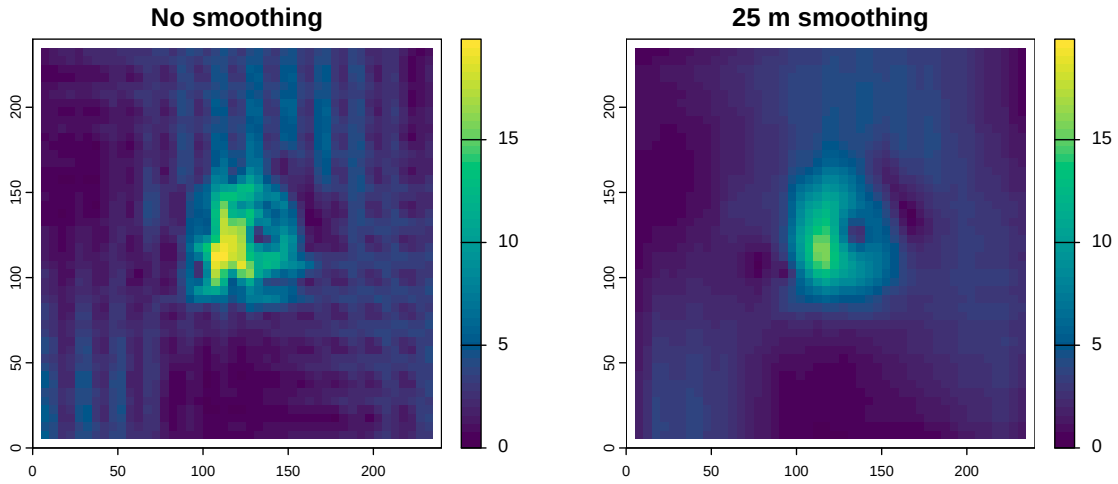


Figure 14: Slope from the same 5 m DEM, before and after 25 m smoothing.

5.3 Compare slope neighbourhoods

The same unsmoothed DEM can use four or eight neighbouring cells for slope and aspect. This parameter changes their calculation; it does not change the fixed eight-neighbour definition of TRI, the TPI radius or the curvature fit.

```
slope.four <- derive_topography(dem = fine[["elevation_m"]], variables = "slope", neighbor

slope.eight <- derive_topography(dem = fine[["elevation_m"]], variables = "slope", neighbo

slope.limit <- max(global(slope.four[["slope_deg"]], "max", na.rm = TRUE)[1, 1],
                   global(slope.eight[["slope_deg"]], "max", na.rm = TRUE)[1, 1])

settings <- par(mfrow = c(1, 2))
plot(slope.four[["slope_deg"]], col = map.colors, range = c(0, slope.limit),
     main = "Four neighbours", mar = map.margins)
plot(slope.eight[["slope_deg"]], col = map.colors, range = c(0, slope.limit),
     main = "Eight neighbours", mar = map.margins)
par(settings)
```

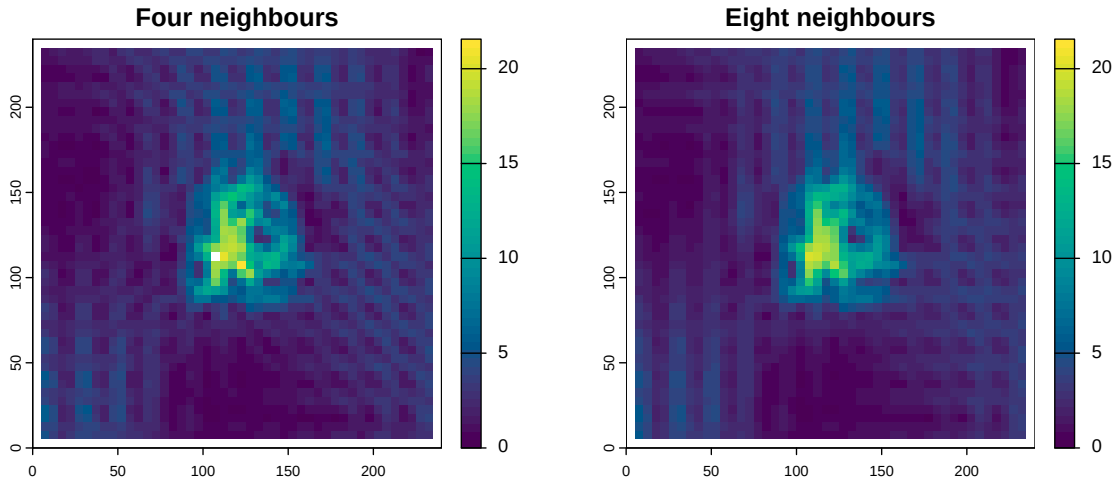


Figure 15: Slope calculated from the same DEM with four or eight neighbours.

6 Keep the layers and save GeoTIFF files

The function returns one `SpatRaster` with descriptive layer names and units. These layers share a grid because they were derived from the same DEM. They may use temporary backing files during the R session. Save a GeoTIFF to retain them: it stores the grid, coordinates and values in a format that other GIS tools can read.

```
writeRaster(topography, file.path(output.dir, "topography.tif"),
            datatype = "FLT4S", gdal = "COMPRESS=LZW", overwrite = TRUE)
write.csv(data.frame(variable = names(topography), unit = units(topography)),
          file.path(output.dir, "topography-layers.csv"), row.names = FALSE)

# Read the saved layers in another R session.
saved.topography <- rast(file.path(output.dir, "topography.tif"))
names(saved.topography)
```

```
[1] "elevation_m"             "slope_deg"
[3] "aspect_deg"              "northness"
[5] "eastness"                "tpi_m"
[7] "tri_m"                   "plan_curvature_per_m"
[9] "profile_curvature_per_m" "contributing_area_m2"
```

```
[11] "twi"                      "drainage_distance_m"
[13] "height_above_drainage_m"
```

```
# Record the software used, including the separate WhiteboxTools program.
R.version.string
```

```
[1] "R version 4.3.1 (2023-06-16)"
```

```
sapply(c("terra", "whitebox"), function(package) as.character(packageVersion(package)))
```

```
   terra whitebox
"1.8.60"  "2.4.0"
```

```
# packageVersion("interp")  # If used for interpolation.
print(whitebox::wbt_version()[1])
```

```
[1] "WhiteboxTools v2.4.0 (c) Dr. John Lindsay 2017-2023"
```

Use a real persistent `output.dir` to keep these files. The local-coordinate example retains a local Cartesian CRS; it is not a geographically positioned map. A real site's GeoTIFF should retain its actual CRS.

The same raster format can later hold soil, canopy and other environmental layers. Those data do not have to share this resolution or grid. Combining them, extracting values and assigning habitats are separate workflows.

References

Detto, M., Muller-Landau, H. C., Mascaro, J., and Asner, G. P. (2013). Hydrological networks and associated topographic variation as templates for the spatial organization of tropical forest vegetation. *PLoS ONE*, 8(10), e76296. https://doi.org/10.1371/journal.pone.0076296

Freund, C. A., Clark, K. E., Curran, J. F., Asner, G. P., and Silman, M. R. (2021). Landslide age, elevation and residual vegetation determine tropical montane forest canopy recovery and biomass accumulation after landslide disturbances in the Peruvian Andes. *Journal of Ecology*, 109, 3555–3571. https://doi.org/10.1111/1365-2745.13737

McNichol, B. H., Wang, R., Hefner, A., Helzer, C., McMahon, S. M., and Russo, S. E. (2024). Topography-driven microclimate gradients shape forest structure, diversity, and composition in a temperate refugial forest. *Plant-Environment Interactions*, 5, e10153. https://doi.org/10.1002/pei3.10153

Muscarella, R., Kolyaie, S., Morton, D. C., Zimmerman, J. K., and Uriarte, M. (2020). Effects of topography on tropical forest structure depend on climate context. *Journal of Ecology*, 108, 145–159. https://doi.org/10.1111/1365-2745.13261

Schietti, J., Emilio, T., Rennó, C. D., Drucker, D. P., Costa, F. R. C., Nogueira, A., Baccaro, F. B., Figueiredo, F., Castilho, C. V., Kinupp, V., Guillaumet, J.-L., Garcia, A. R. M., Lima, A. P., and Magnusson, W. E. (2014). Vertical distance from drainage drives floristic composition changes in an Amazonian rainforest. *Plant Ecology & Diversity*, 7(1–2), 241–253. https://doi.org/10.1080/17550874.2013.783642

Zhang, B., Jackson, T. D., Coomes, D. A., Burslem, D. F. R. P., Nilus, R., Bittencourt, P. R. L., Bartholomew, D. C., Rowland, L., Fischer, F. J., and Jucker, T. (2025). Soils and topography drive large and predictable shifts in canopy dynamics across tropical forest landscapes. *New Phytologist*, 247, 1666–1679. https://doi.org/10.1111/nph.70300

Software documentation: terra, interp, and whitebox. Curvature definitions were checked against the WhiteboxTools 2.4.0 plan-curvature source and profile-curvature source.